

# CS3212 Algorithms

Aya Zirikly  
09/15/2026

# Maintaining Heap Property: MAX-HEAPIFY

**Purpose:** Corrects a single heap violation at node  $i$ , assuming subtrees rooted at  $\text{LEFT}(i)$  and  $\text{RIGHT}(i)$  already satisfy the Max-Heap property.

## Algorithm Logic:

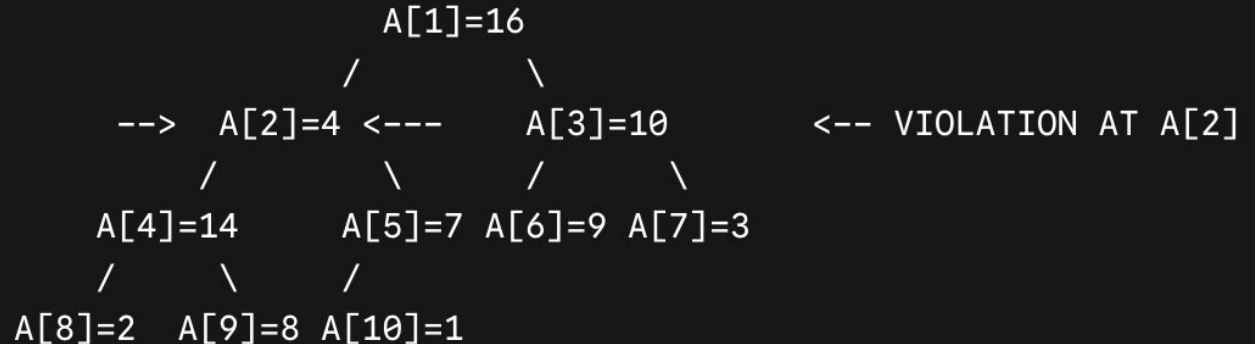
1. Compute left child  $L = \text{LEFT}(i)$  and right child  $R = \text{RIGHT}(i)$ .
2. Compare  $A[i]$ ,  $A[L]$ , and  $A[R]$  to locate index of maximum value (**largest**).
3. If  $\text{largest} \neq i$ :
  - Swap  $A[i]$  with  $A[\text{largest}]$ .
  - Recursively invoke  $\text{MAX-HEAPIFY}(A, \text{largest})$  to trickle value down.

# MAX-HEAPIFY Example

Input Array: A = [16, 4, 10, 14, 7, 9, 3, 2, 8, 1]

```
Array: [16, 4, 10, 14, 7, 9, 3, 2, 8, 1]
      |  |  |  |  |  |  |  |  |  |
Indices: 1  2  3  4  5  6  7  8  9 10
```

MAX-HEAPIFY(A, 2)



# MAX-HEAPIFY Example

Input Array:  $A = [16, 4, 10, 14, 7, 9, 3, 2, 8, 1]$ , Call: MAX-HEAPIFY( $A, 2$ )

- $A[2] = 4$
- Children of index 2: LEFT(2) =  $A(i*2=4) = 14$
- RIGHT(2) =  $A(i*2+1=5) = 7$
- $\max(4, 14, 7) = 14$  at index 4.
- Swap  $A[2]$  (4) and max =  $A[4] = 14 \rightarrow$  Array:  $[16, 14, 10, 4, 7, 9, 3, 2, 8, 1]$ .

# MAX-HEAPIFY Example

Input Array:  $A = [16, 4, 10, 14, 7, 9, 3, 2, 8, 1]$ , Call: MAX-HEAPIFY( $A, 2$ )

- Recursive Call MAX-HEAPIFY( $A, 4$ ):
  - Children of index 4:  $A[8] = 2, A[9] = 8$
  - Find max:  $\max(4, 2, 8) = 8$  at index 9.
  - Swap  $A[4]$  (4) and  $A[9]$  (8)  $\rightarrow$  Array:  $[16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$ .



Heap property restored at all downstream subtrees.

*Every parent node satisfies the max-heap property*

# MAX-HEAPIFY Complexity —Recurrence Derivation

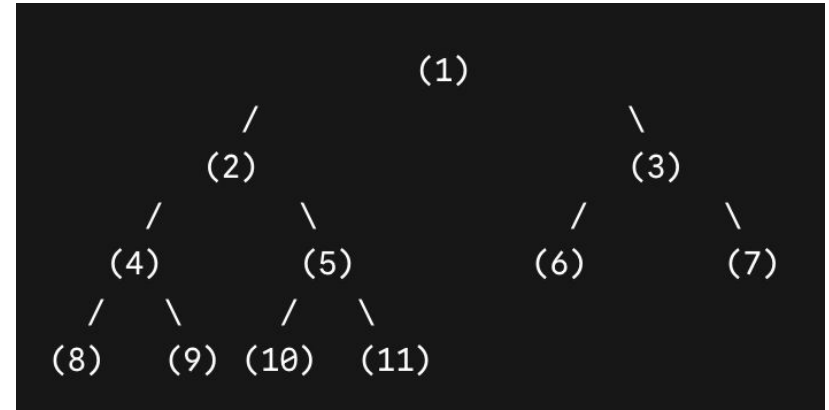
**Operation Cost per Node:** Comparing the parent node with its left and right children and performing a swap takes constant time  $O(1)$ .

**Worst-Case Subproblem Bound ( $2n/3$ ):**

- A heap is a nearly complete binary tree filled level by level (level order traversal)
- **Worst Case Geometry:** the bottom row of the tree is **exactly half full**.
- Under this layout, the left subtree is completely filled while the right subtree is missing its bottom level.
- The larger subtree contains at most  $2n/3$  of the total nodes in the tree.

- **Total Nodes (n):** 11
- **Left Subtree Count:** 7 nodes ([2, 4, 5, 8, 9, 10, 11])
- **Right Subtree Count:** 3 nodes ([3, 6, 7])
- **Root Node:** 1 node ([1])

**Larger Subtree Fraction =  $7 / 11 \approx 63.6\% \leq 66.7\%$  (2 / 3)**

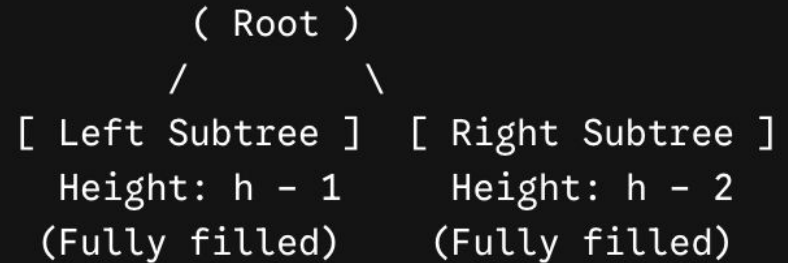


# MAX-HEAPIFY Complexity —Recurrence Derivation

**Left Subtree:** Completely full down to height  $h - 1$ .

**Right Subtree:** Completely full only down to height  $h - 2$   
(it contains zero nodes on the bottom level).

Full binary tree of height  $k$  contains?



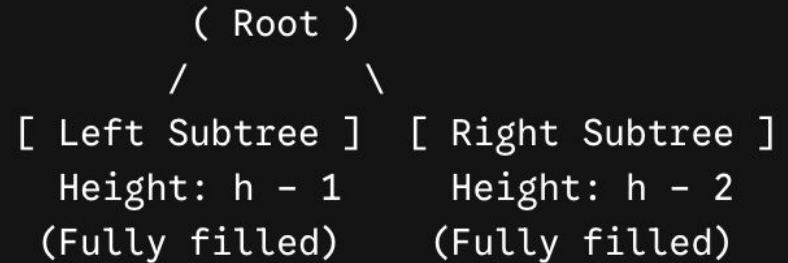
# MAX-HEAPIFY Complexity — Recurrence Derivation

**Left Subtree:** Completely full down to height  $h - 1$ .

**Right Subtree:** Completely full only down to height  $h - 2$   
(it contains zero nodes on the bottom level).

Full binary tree of height  $k$  contains

$$2^{(k+1)} - 1 \text{ nodes}$$



# MAX-HEAPIFY Complexity — Recurrence Derivation

full binary tree of height  $k$  contains  $2^{(k+1)} - 1$  nodes

**Left Subtree ( $n_{\text{left}}$ ):**

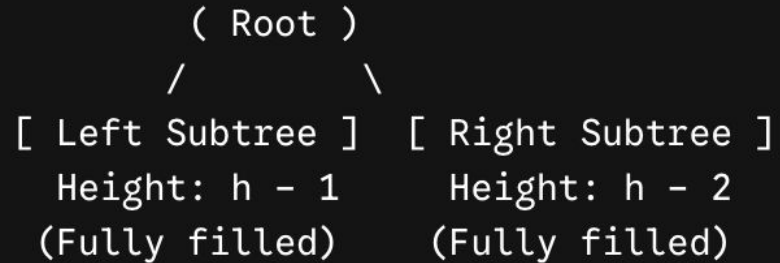
$$n_{\text{left}} = 2^{((h-1)+1)} - 1 = 2^h - 1$$

**Right Subtree ( $n_{\text{right}}$ ):**

$$n_{\text{right}} = 2^{((h-2)+1)} - 1 = 2^{(h-1)} - 1$$

**Total Nodes in Tree ( $n$ ):**

Root (1) + Left Subtree + Right Subtree:



# MAX-HEAPIFY Complexity —Recurrence Derivation

**Left Subtree ( $n_{\text{left}}$ ):**  $2^h - 1$

**Right Subtree ( $n_{\text{right}}$ ):**  $2^{(h-1)} - 1$

**Total Nodes in Tree ( $n$ ):**

Root (1) + Left Subtree + Right Subtree:

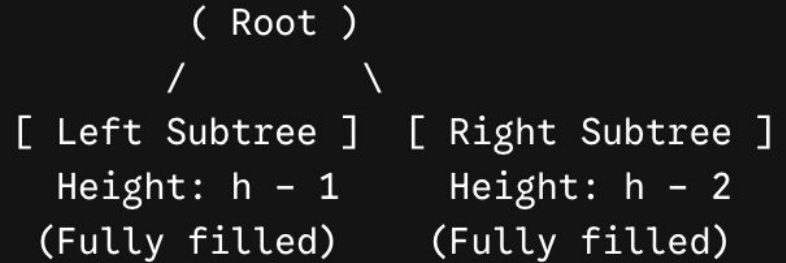
$$n = 1 + n_{\text{left}} + n_{\text{right}}$$

$$n = 1 + (2^h - 1) + (2^{(h-1)} - 1)$$

$$n = 2^h + 2^{(h-1)} - 1$$

$$n = 2 * 2^{(h-1)} + 1 * 2^{(h-1)} - 1$$

$$n = 3 * 2^{(h-1)} - 1$$



# Comparing Subtree Sizes

- **Left Subtree:** Holds a complete row of leaves at the deepest level.
- **Right Subtree:** Completely misses the deepest level.
- **Result:** The left subtree is larger than the right subtree, but **never exceeds  $(\frac{2}{3})n$**  (66.7%) of the total nodes.

# Connecting Tree Geometry to Algorithm Speed

## How **MAX-HEAPIFY** Traverses the Tree:

1. Compares root with direct children  $\Theta(1)$ .
2. Swaps root with largest child if needed.
3. **Recurse down into ONLY ONE child subtree.**

## Worst-Case Subproblem Size:

- In the worst case, **MAX-HEAPIFY** recurses into the larger (left) child subtree.
- Input size drops from  $n$  to at most  $(\frac{2}{3})n$ .

## Formulating the Recurrence:

$$T(n) \leq T(2n / 3) + \Theta(1)$$

# MAX-HEAPIFY Complexity —Recurrence Derivation

$$T(n) \leq T(2n / 3) + \theta(1)$$

- $T(2n / 3)$ : Maximum time spent recursing down the larger child subtree.
- $\theta(1)$ : Non-recursive work (comparing values and swapping array elements).

# MAX-HEAPIFY Complexity — Solving the Recurrence

**Equation:**  $T(n) = 1 * T(n / (3/2)) + \theta(1)$

**Parameters:**  $a = 1, b = 3/2$  (or 1.5),  $f(n) = \theta(1)$

**Calculate Leaf Work:**  $n^{(\log_b a)} = n^{(\log_{(1.5)} 1)} = n^0 = 1$

**Compare with  $f(n)$ :**  $f(n) = \theta(1) = \theta(n^0 * (\log n)^0)$

*(Matches Case 2 of Master Theorem with  $k = 0$ )*

# How it matches case 2 Master Theorem

The general formulation for Case 2 of the Master Theorem applies when  $f(n)$  matches  $n^{\log_b a}$  up to a logarithmic power:

$$f(n) = \Theta(n^{\log_b a} * (\log n)^k) \text{ for some } k \geq 0$$

$$n^{\log_b a} = 1 \text{ and } f(n) = \Theta(1):$$

$$\Theta(1) = \Theta(1 * (\log n)^k)$$

$$(\log n)^k = 1 \Rightarrow k = 0 \text{ (since } (\log n)^0 = 1)$$

$$\Rightarrow f(n) = \Theta(n^{\log_b a})$$

# How it matches case 2 Master Theorem

❑  $f(n)$  matches the leaf work capacity ( $\Theta(1)$  per level)  $\Rightarrow$

work is evenly distributed across all  $\Theta(\log n)$  levels of the tree.

# MAX-HEAPIFY Complexity — Solving the Recurrence

**Work Per Level:** Constant non-recursive work  $O(1)$  per level.

**Heap Height:** A heap of  $n$  nodes has height  $h = \text{floor}(\log_2 n)$ .

**Total Work Calculation:**

Total Work = (Work per level) \* (Tree height)

Total Work =  $O(1) * \text{floor}(\log_2 n) = O(\log n)$

# Building a Heap: BUILD-MAX-HEAP

Goal: Convert unsorted array  $A[1..n]$  into a valid max-heap.

Key Observation:

- Elements in array slice  $A[\text{floor}(n/2)+1 .. n]$  are all leaves (1-element heaps).
- Leaf nodes satisfy the Max-Heap property!
- Process remaining non-leaf nodes in reverse level order ( $\text{floor}(n/2)$  down to 1).

Algorithm Procedure:

**BUILD-MAX-HEAP(A):**

**heap-size[A] = length[A]**

**for i = floor(length[A] / 2) down to 1:**

**MAX-HEAPIFY(A, i)**

# Building a Heap: BUILD-MAX-HEAP

Goal: Convert unsorted array  $A[1..n]$  into a valid max-heap.

Key Observation:

- Elements in array slice  $A[\text{floor}(n/2)+1 .. n]$  are all leaves (1-element heaps).
- Leaf nodes trivially satisfy the Max-Heap property!

Algorithm Procedure:

**BUILD-MAX-HEAP(A):**

**heap-size[A] = length[A]**

**for i = floor(length[A] / 2) down to 1:**

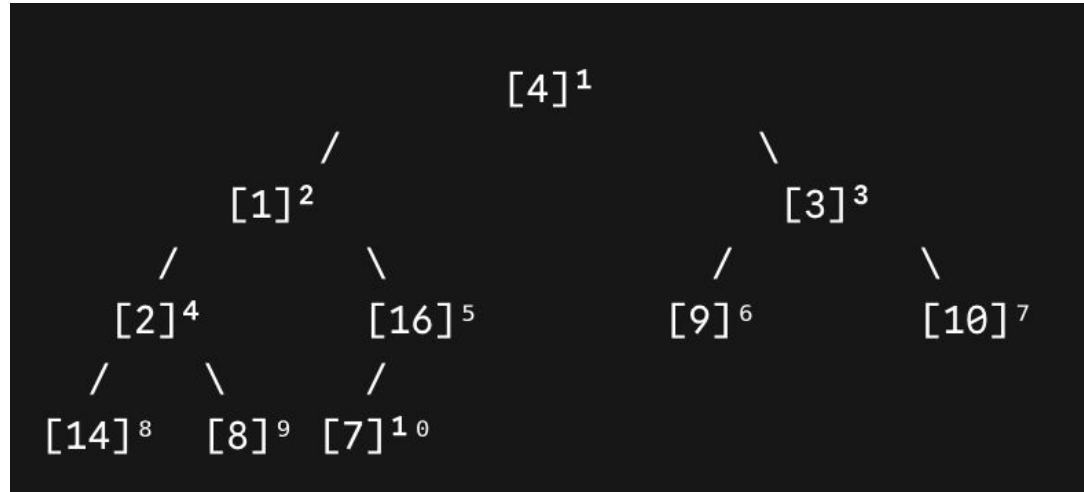
**MAX-HEAPIFY(A, i)**

## Why iterate backward (down to 1)?

This establishes a strict **bottom-up order**. By fixing lower subtrees first, it guarantees that whenever **MAX-HEAPIFY(A, i)** is called on node  $i$ , the left and right subtrees rooted at  $i$ 's children are already valid max-heaps—fulfilling **MAX-HEAPIFY**'s required precondition.

# BUILD-MAX-HEAP Trace

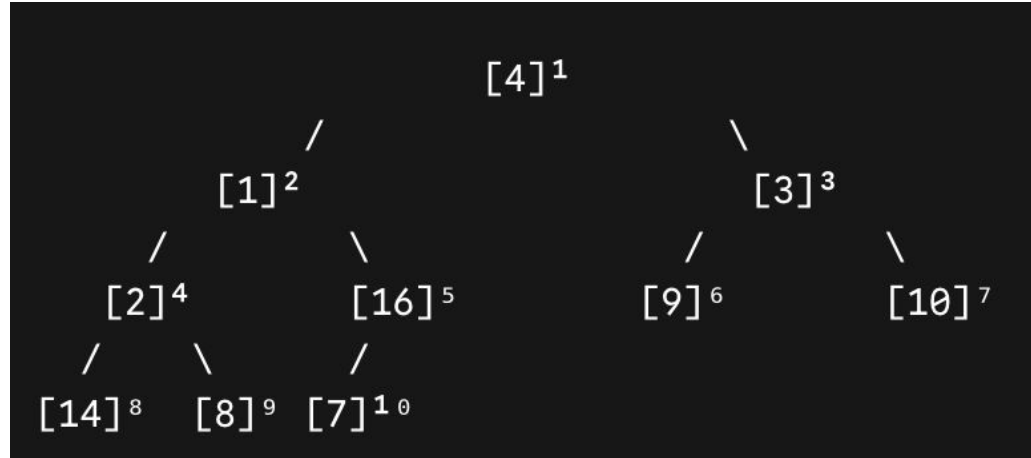
Input Array: [4, 1, 3, 2, 16, 9, 10, 14, 8, 7], n = 10



**Step 1:  $i = 5$  (Value = 16)**

- **Children:**  $\text{LEFT}(5) = \text{index } 10$  (value 7).
- **Compare:**  $16 \geq 7 \rightarrow$  Already satisfies Max-Heap property!
- **Action:** No swap needed.

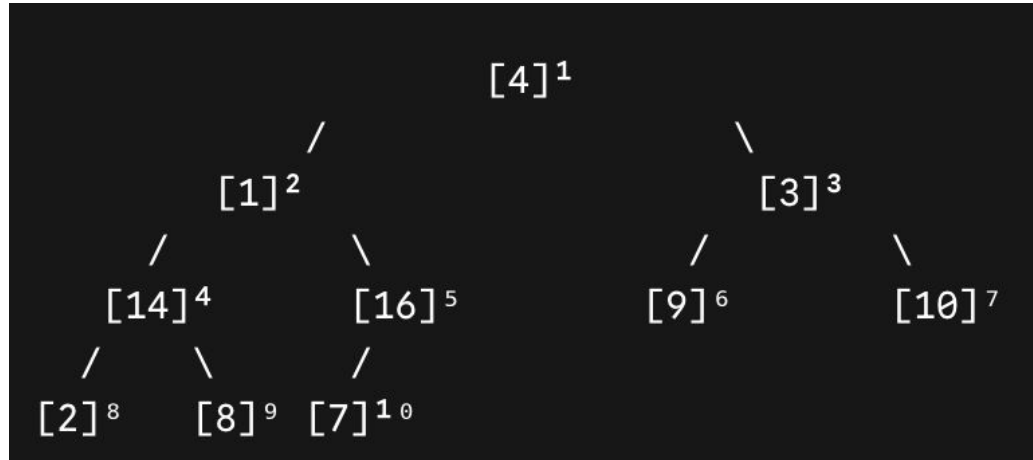
Array: [4, 1, 3, 2, 16, 9, 10, 14, 8, 7]



Step 2: **i = 4** (Value = 2)

- **Children:** **LEFT(4)** = index 8 (value 14), **RIGHT(4)** = index 9 (value 8).
- **Compare:** Max child is **14** at index 8.
- **Action:** Swap **2** and **14**.

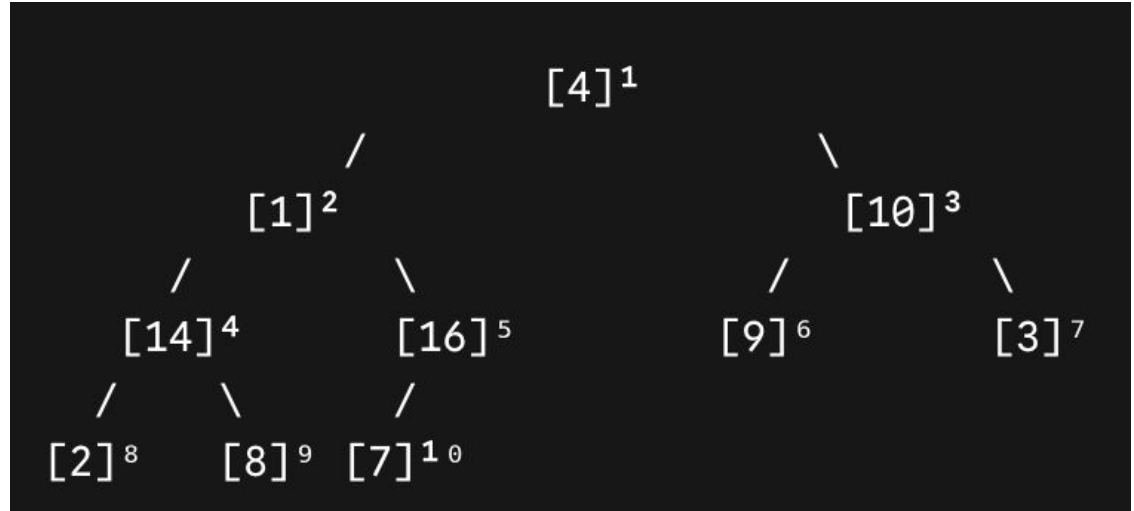
Array: [4, 1, 3, 14, 16, 9, 10, 2, 8, 7]



Step 3: **i = 3** (Value = 3)

- **Children:** **LEFT(3)** = index 6 (value 9), **RIGHT(3)** = index 7 (value 10).
- **Compare:** Max child is **10** at index 7.
- **Action:** Swap **3** and **10**.

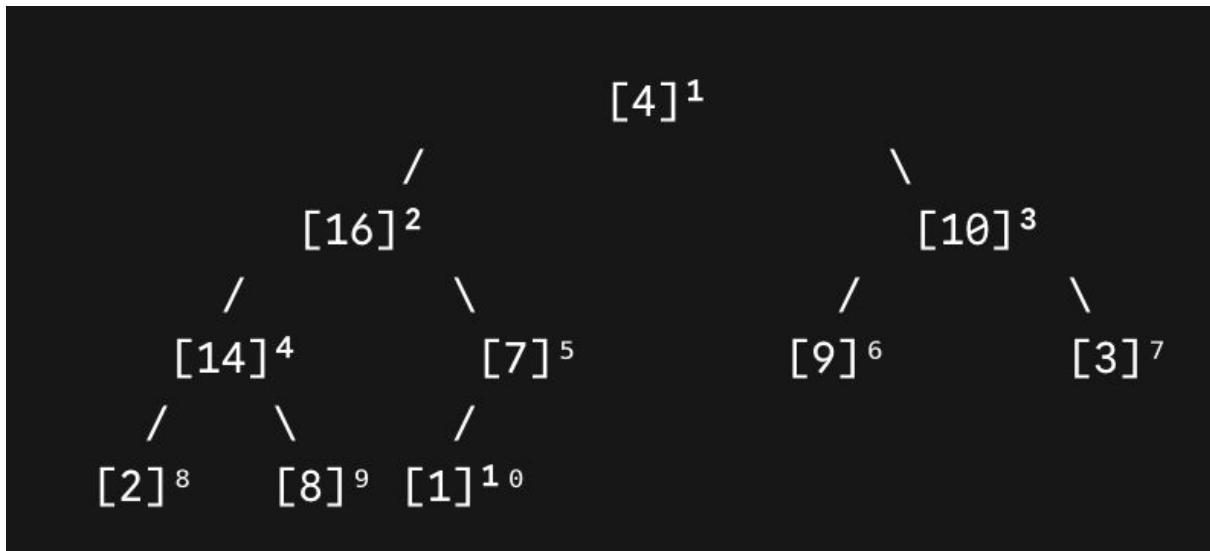
Array: [4, 1, 10, 14, 16, 9, 3, 2, 8, 7]



**Step 4:  $i = 2$  (Value = 1)**

- **Children:**  $\text{LEFT}(2) = \text{index } 4$  (value 14),  $\text{RIGHT}(2) = \text{index } 5$  (value 16).
- **Compare:** Max child is **16** at index 5  $\rightarrow$  Swap **1** and **16**.
- **Trickle Down:** At index 5, **1** is compared with its child at index 10 (value 7)  $\rightarrow$  Swap **1** and **7**.

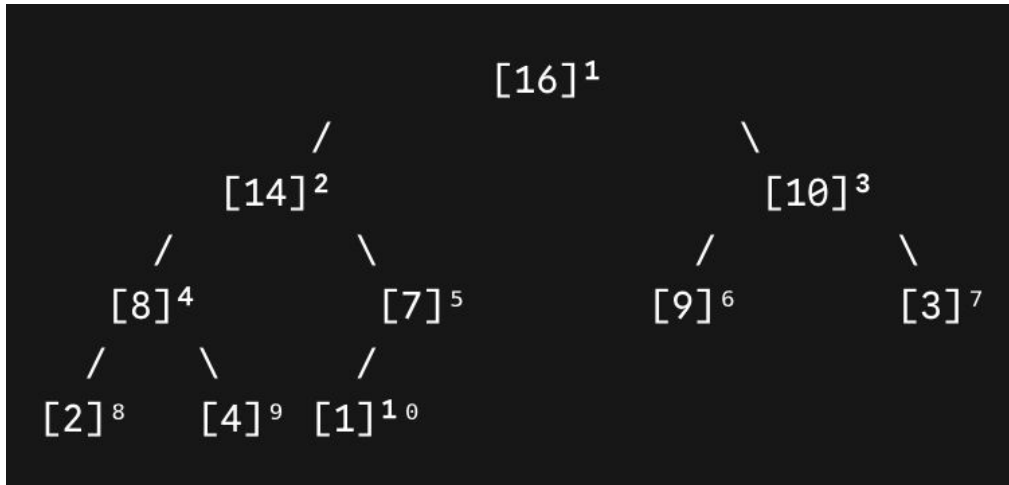
**Array:** [4, 16, 10, 14, 7, 9, 3, 2, 8, 1]



**Step 5:  $i = 1$  (Value = 4)**

- **Compare Root:** Max child is **16** at index 2 → Swap **4** and **16**.
- **Trickle Down (Level 1):** At index 2, **4** is compared with children (14, 7) → Swap **4** with **14** at index 4.
- **Trickle Down (Level 2):** At index 4, **4** is compared with children (2, 8) → Swap **4** with **8** at index 9.

**Resulting Max-Heap Array:** [16, 14, 10, 8, 7, 9, 3, 2, 4, 1]



$n$  calls to MAX-HEAPIFY taking  $O(\log n)$  time yields  $O(n \log n)$

**But why Isn't BUILD-MAX-HEAP  $O(n \log n)$ ?**

# Why this bound is loose?

- ❖  $O(\log n)$  is the cost for a node at the **root** (height  $h = \log n$ ).
- ❖ Most nodes live near the **bottom** of the tree where height  $h$  is very small!
- ❖ ~**50%** of nodes are leaves (height  $0 \rightarrow 0$  swaps needed).
- ❖ ~**25%** of nodes are 1 level above leaves (height  $1 \rightarrow$  at most  $1$  swap needed).
- ❖ Only **1** node lives at the top (height  $\log n \rightarrow$  up to  $\log n$  swaps needed).

# Total work

## Cost Per Level:

- Work at height  $h$ :  $O(h)$  operations per node.
- Max nodes at height  $h$ :  $\lceil n / 2^{h+1} \rceil$

**Summing Work Across All Heights:** Total Work =  $\sum$  (from  $h = 0$  to  $\lfloor \log n \rfloor$ )  $\lceil n / 2^{h+1} \rceil \cdot O(h)$

## Simplifying the Expression:

- Pull out constant factor  $n$  and big-O notation:

$$\text{Total Work} = O(n \cdot \sum \text{(from } h = 0 \text{ to } \lfloor \log n \rfloor \text{)} h / 2^h)$$

- Upper-bound the finite sum by extending it to infinity ( $h = 0$  to  $\infty$ ):

$$\text{Total Work} \leq O(n \cdot \sum \text{(from } h = 0 \text{ to } \infty \text{)} h / 2^h)$$

# Solving the infinite series

**The Infinite Series:**  $S = \sum (\text{from } h = 0 \text{ to } \infty) h / 2^h = (0/1) + (1/2) + (2/4) + (3/8) + (4/16) + \dots$

**Visualizing the Sum:**

- Line 1:  $(1/2) + (1/4) + (1/8) + (1/16) + \dots = 1$
- Line 2:  $(1/4) + (1/8) + (1/16) + \dots = 1/2$
- Line 3:  $(1/8) + (1/16) + \dots = 1/4$
- Line 4:  $(1/16) + \dots = 1/8$
- **Total Sum S:**  $1 + 1/2 + 1/4 + 1/8 + \dots = 2$

**Algebraic Identity:**  $\sum (\text{from } h = 0 \text{ to } \infty) h / 2^h = 2$

Total Work =  $O(n \cdot \sum (\text{from } h = 0 \text{ to } \infty) h / 2^h)$  Total Work =  $O(n \cdot 2)$

**Total Work =  $O(n)$**

# Key takeaways

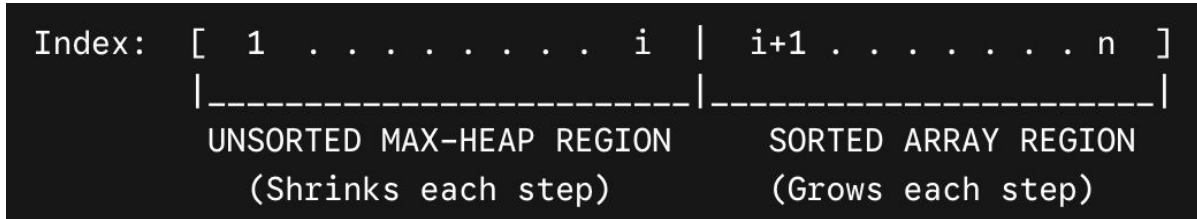
- ❑ **BUILD-MAX-HEAP** runs in linear time  $O(n)$ .
- ❑ Building a heap from an unsorted array takes  $O(n)$  time

# The HeapSort Algorithm

## The intuition

Convert an unsorted array into a Max-Heap, then repeatedly pull the largest element from the top and place it at the end of the array.

**Array Division:** As the algorithm runs, the array is partitioned into two dynamic regions:

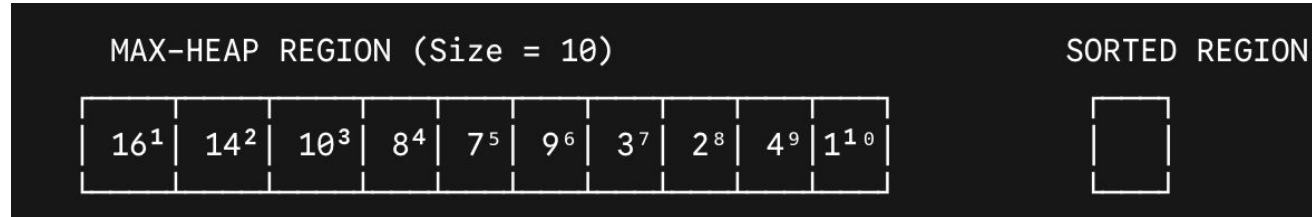


# The HeapSort Algorithm

```
HEAPSORT(A):
```

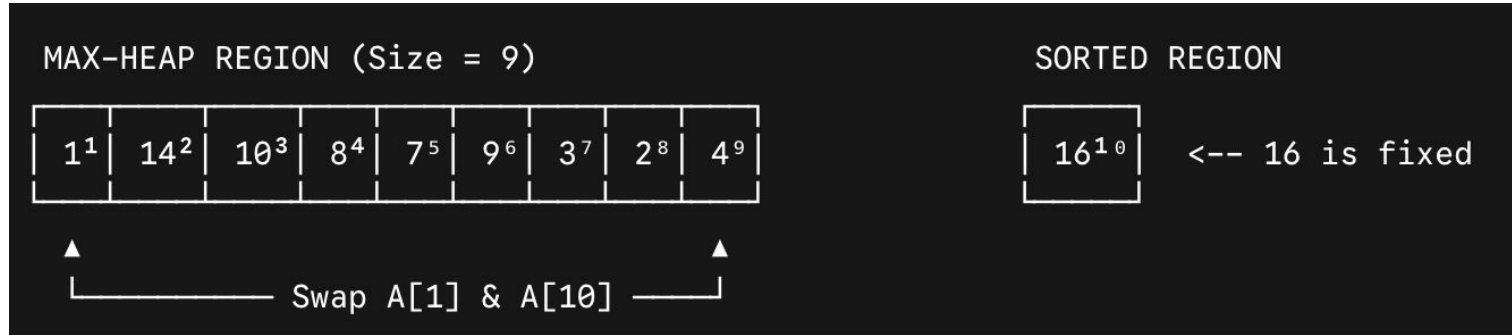
```
    BUILD-MAX-HEAP(A)           // Phase 1: Prep array in  $O(n)$  time  
    for i = length[A] down to 2: // Phase 2: Extract max  $(n-1)$  times  
        Swap A[1] with A[i]      // 1. Move root (max) to sorted end  
        heap-size[A] = heap-size[A] - 1 // 2. Shrink heap boundary  
        MAX-HEAPIFY(A, 1)        // 3. Fix root in  $O(\log n)$  time
```

# Initial State (After BUILD-MAX-HEAP)



# Step 1: Iteration, $i = 10$

Swap Root  $A[1]$  with  $A[10]$  & Shrink Heap Boundary:



# Step 1: Iteration, $i = 10$

Call **MAX-HEAPIFY(A, 1)** (float / sift 1 down):

- Swap 1 with 14 at index 2
- Swap 1 with 8 at index 4
- Swap 1 with 4 at index 9

REPAIRED MAX-HEAP (Size = 9)

|                 |                |                 |                |                |                |                |                |                |
|-----------------|----------------|-----------------|----------------|----------------|----------------|----------------|----------------|----------------|
| 14 <sup>1</sup> | 8 <sup>2</sup> | 10 <sup>3</sup> | 4 <sup>4</sup> | 7 <sup>5</sup> | 9 <sup>6</sup> | 3 <sup>7</sup> | 2 <sup>8</sup> | 1 <sup>9</sup> |
|-----------------|----------------|-----------------|----------------|----------------|----------------|----------------|----------------|----------------|

SORTED REGION

|                  |
|------------------|
| 16 <sup>10</sup> |
|------------------|

## Step 2: Iteration $i = 9$

Swap Root  $A[1]$  with  $A[9]$  & Shrink Heap Boundary:

MAX-HEAP REGION (Heap Size = 8)

|       |       |        |       |       |       |       |       |
|-------|-------|--------|-------|-------|-------|-------|-------|
| $1^1$ | $8^2$ | $10^3$ | $4^4$ | $7^5$ | $9^6$ | $3^7$ | $2^8$ |
|-------|-------|--------|-------|-------|-------|-------|-------|

SORTED REGION

|        |           |
|--------|-----------|
| $14^9$ | $16^{10}$ |
|--------|-----------|

Call **MAX-HEAPIFY(A, 1)** (float / sift 1 down):

- Swap **1** with **10** at index 3
- Swap **1** with **9** at index 6

REPAIRED MAX-HEAP (Heap Size = 8)

|                 |                |                |                |                |                |                |                |
|-----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|
| 10 <sup>1</sup> | 8 <sup>2</sup> | 9 <sup>3</sup> | 4 <sup>4</sup> | 7 <sup>5</sup> | 1 <sup>6</sup> | 3 <sup>7</sup> | 2 <sup>8</sup> |
|-----------------|----------------|----------------|----------------|----------------|----------------|----------------|----------------|

SORTED REGION

|                 |                  |
|-----------------|------------------|
| 14 <sup>9</sup> | 16 <sup>10</sup> |
|-----------------|------------------|

# HeapSort Complexity & Trade-offs —Time Complexity

| Metric          | Complexity         |
|-----------------|--------------------|
| Best Case       | $\Theta(n \log n)$ |
| Average Case    | $\Theta(n \log n)$ |
| Worst Case      | $\Theta(n \log n)$ |
| Auxiliary Space | $O(1)$             |
| Stability       | Not Stable         |

## Stability: NOT Stable

Long-distance swaps between root  $A[1]$  and end element  $A[i]$  break the relative ordering of duplicate keys.

# HeapSort Complexity & Trade-offs —Time Complexity

## HeapSort vs. QuickSort in Practice

- **Guaranteed vs. Expected Performance:**
  - **HeapSort:** Hard  $\Theta(n \log n)$  worst-case guarantee; no worst-case input triggers quadratic time.
  - **QuickSort:**  $O(n \log n)$  average case, but risks worst-case  $O(n^2)$  without proper pivot choices.